

# API Design, Authentication, and Authorization

A good API defines behavior under valid requests, invalid input, retries, and unauthorized access. Authentication establishes identity; authorization decides permitted actions on a resource. Neither a signed token nor a hidden frontend control automatically enforces object-level access.

**For:** Backend and full-stack engineers who want to build and explain safe API contracts, identity checks, resource authorization, retries, pagination, and input validation.

Suggested pace: 3 weeks of defensive API practice | Web service fundamentals

## What you will be able to do

- Define stable resource contracts, errors, pagination, and compatibility behavior.
- Separate authentication, authorization, and transport protection.
- Use server-side checks and least privilege for every sensitive resource operation.
- Handle idempotency, validation, and abuse limits without trusting client state.

## Build the right mental model

### The server owns the policy

A client-supplied role, price, tenant ID, or paid flag is input to validate, not authority. Resolve identity through the accepted authentication mechanism and enforce resource ownership and permissions on the server. UI visibility is not an access-control boundary.

### Tokens have a verification contract

Validate the expected issuer, audience, signature, expiry, and relevant claims according to the chosen protocol and library. A JWT being decodable does not mean it is valid. Revocation and session lifetime depend on the application's threat and operational model.

### APIs evolve through compatibility

Specify optional versus required fields, stable identifiers, error structure, and pagination order. An idempotency key should bind to the logical operation and payload. A versioning strategy cannot compensate for silently changing a documented response meaning.

# Study with evidence.

Use a local service and synthetic users with different permissions. Write authorization tests before adding UI controls. Rely on established authentication libraries and provider documentation rather than implementing cryptographic protocols from scratch.

---

## Week 1: Contract and validation

Define resources, methods, errors, required fields, size limits, and stable pagination. Validate types and business constraints. Distinguish malformed requests, absent resources, conflicts, and unauthorized operations according to the API policy.

**Deliverable and check:** An API specification and tests for valid, invalid, duplicate, and boundary requests.

---

## Week 2, first half: Identity and sessions

Integrate an established authentication approach in a test environment. Understand token verification, session expiry, secure transport, and secret storage. Keep credentials out of source, URLs, and logs.

**Deliverable and check:** A clear identity flow with expired, malformed, and wrong-audience cases rejected.

---

## Week 2, second half: Resource authorization

Test two users and two tenants accessing each other's objects. Apply authorization to reads, writes, downloads, and background operations. Define administrative exceptions narrowly and audit sensitive actions.

**Deliverable and check:** An object-level permission test suite that does not rely on the frontend hiding buttons.

---

## Week 3: Retries and abuse resistance

Add payload-bound idempotency for selected writes, rate and size limits, safe errors, and audit metadata. Review dependency failure and partial completion. Document key rotation and incident response at an appropriate level.

**Deliverable and check:** A defensive review with specific boundaries, tests, and remaining risks.

---

### A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

# Apply it. Explain it.

## Protect an order lookup and retryable creation

A user can create an order and fetch it by ID. The service must prevent one user from reading another's order and avoid creating two orders when a response is lost and the client retries.

### A worked approach

1. Authenticate the caller and scope every order lookup to the permitted owner or tenant, using an authoritative server-side policy. Do not trust an owner field supplied by the client or assume an unpredictable ID is sufficient authorization.
2. For creation, bind a stable idempotency key to the authenticated scope and request hash. Persist the operation result with the order transaction. A repeated key with a different payload should produce a defined conflict rather than reuse an unrelated result.
3. Test user A reading user B's order, expired credentials, duplicate creation, mismatched payload reuse, and a simulated lost response. Log identifiers and outcomes without storing payment details or tokens in ordinary logs.

#### Expected result

The order's ownership and logical creation identity are enforced at the server and data boundary. Client controls improve usability but are not relied upon to prevent unauthorized access or duplicate effects.

### Verify the result

- Are resource permissions checked on every relevant operation?
- Is the idempotency key scoped and payload-bound?
- Do logs and error messages avoid leaking secrets or private records?

### Extend the exercise

Add an order export background job. Ensure the worker preserves the requester's authorized scope and that the download link cannot bypass current access policy or outlive its intended validity.

# Make the learning stick.

## Is authentication enough to secure an API?

No. An authenticated user may still be unauthorized for a particular object or action. Validate resource-level permissions, input, limits, and business invariants on the server, including background and download paths.

## Does a JWT eliminate server-side state?

It can carry signed claims, but revocation, sessions, permissions, and business data may still require authoritative state. The right design depends on lifetime and policy requirements rather than a blanket statelessness claim.

## Why not implement my own login cryptography?

Authentication and cryptographic protocols have subtle requirements and failure modes. Use maintained libraries and established standards, configure them correctly, and test the application authorization boundaries that those libraries do not decide for you.

## Avoid these traps

- Treating client-side flags or hidden controls as authorization.
- Accepting a decoded token without full validation.
- Applying security checks only to interactive endpoints and forgetting jobs or downloads.

## Ready for the next step?

- Identity and resource permissions are separate and tested.
- Sensitive operations enforce policy server-side.
- Retries have scoped identities and defined conflicts.
- Secrets and private data are excluded from public artifacts and logs.

## Reliable next reads

- [OWASP API Security Project](#)
- [OWASP authentication guidance](#)
- [MDN HTTP methods](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.